Ders 12 Genişletilmiş Notu: HITS Algoritması ve Web Kullanım Madenciliği

Ders 12 Genişletilmiş Notu: HITS Algoritması ve Web Kullanım Madenciliği

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bağlantı Analizinin Kısa Tekrarı
- 2. PageRank'in Sorgudan Bağımsızlığı ve Bir Eleştiri
- 3. HITS Algoritması: Kök Küme ve Taban Küme
- 4. Otorite ve Merkez (Hub) Kavramları
- 5. HITS Denklemleri ve Matris Gösterimi
- 6. Kuvvet Yinelemesi ile Çözüm ve Ortak Atıf İlişkisi
- 7. HITS'in Avantajları ve Dezavantajları
- 8. Web Kullanım Madenciliğine Giriş
- 9. Veri Kaynakları ve Tıklama Akışı Verisi
- 10. Kullanıcı İşlem Veritabanı ve Veri Hazırlama Süreci
- 11. Bölüm (Episode) Tanımlama
- 12. Veri Temizleme
- 13. Kullanıcı Tanımlama
- 14. Oturum Tanımlama ve Sezgisel Yöntemler
- 15. Sayfa Görüntüleme Tanımlama ve Yol Tamamlama
- 16. Olaylar (Events) ve Tıklama Akışı
- 17. Örüntü Keşfi, Mimari ve Kullanım Alanları

1. Bağlantı Analizinin Kısa Tekrarı

Hoca derse, yılın son gününde bağlantı analizi (link analysis) konusunu bitirip web kullanım madenciliğine (web usage mining) geçeceklerini söyleyerek başladı. Bu ders büyük ölçüde Türkçe anlatıldı.

Önceki hafta sosyal ağ analizi (social network analysis) ve bağlantı analizi konuşulmuştu. Bu kapsamda görülen metrikler şunlardı:

- **Merkezilik (centrality)** ve **prestij (prestige)** metrikleri.
- **Ortak atıf (co-citation):** Özellikle içinde kaynakça bölümü olan belgelerin birbirine benzerliğini bulmak için kullanılır.
- **Bibliyografik eşleşme (bibliographic coupling).**
- **PageRank:** Nasıl hesaplandığı ve algoritmanın nasıl çalıştığı.

Hoca PageRank'in prestij metrikleri içindeki sıra prestiji (rank prestige) üzerine kurulduğunu hatırlattı. PageRank, web alanında karşılaşılan çıkmaz sayfa (dead end) ve örümcek tuzağı (spider trap) gibi sorunların üstesinden gelebiliyordu. Bu durumlara rağmen her düğümün, yani her web sayfasının popülerlik değerini bulabiliyordu.

2. PageRank'in Sorgudan Bağımsızlığı ve Bir Eleştiri

Bu derste PageRank'in alternatifi olan HITS algoritması anlatıldı. HITS, Hypertext Induced Topic Search'ün kısaltmasıdır. Hoca iki algoritmanın da 1998'de ortaya çıktığını söyledi. PageRank Stanford Üniversitesi'nden, HITS ise Cornell Üniversitesi'nden çıktı. Hocaya göre PageRank ilk olarak Google tarafından kullanıldı ve zaten Google'ın ortaya çıkmasını sağlayan nedenlerden biriydi. HITS'i ise Yahoo kullanmaya başladı.

> **Ders sonrası not:** Hoca HITS'in Yahoo tarafından kullanıldığını söyledi. HITS'i büyük olasılıkla Jon Kleinberg yayımladı. Ticari kullanımda en çok bilinen örnek büyük olasılıkla Teoma arama motorudur (sonradan Ask.com'un parçası oldu). Yahoo'nun HITS kullandığına dair yaygın bir kaynak büyük olasılıkla yoktur.

PageRank anlatılırken hesap hep sayfaların birbirine verdiği köprü bağlantıları (hyperlink) üzerinden yapılmıştı. Kullanıcının verdiği bir anahtar kelimeye göre PageRank skoru hesaplamaktan hiç söz edilmemişti. Bu nedenle PageRank **sorgudan bağımsızdır (query independent)**: Sorgu ne olursa olsun sayfaların PageRank skoru önceden hesaplanabilir.

Hoca bu özelliğe bir dezavantaj olarak da bakılabileceğini söyledi. Kullanıcının "web data mining" aradığını düşünelim. Bu kelimeleri içeren çok sayıda sayfa bulunur ve sonra bu sayfalar PageRank'e göre yeniden sıralanır (re-rank). Normalde bu üç kelime için en otoriter sayfa, başlığı "Web Data Mining" olan bir kitabın sayfasıdır. Bu sayfayı ilk 10 sonuçta göstermemek en mantıksız şey olur. Ama PageRank anahtar kelimeden bağımsız yeniden sıralama yaptığı için, yine bu kelimeleri içeren ama o kadar otoriter olmayan, buna karşın popüler, yani PageRank değeri yüksek sayfalar öne çıkabilir. Popülerliği düşük ama o anahtar kelime için en otoriter olan sayfa ilk 10 sonuca girmeyebilir. Bu, PageRank'e getirilen eleştirilerden biridir.

HITS bu soruna bir çözüm getirir: Bağlantı analizini **sorguya bağlı (query dependent)** olarak yapar.

3. HITS Algoritması: Kök Küme ve Taban Küme

3.1 Ortak ilk adım: ters dizinde arama

Hoca, PageRank de kullanılsa HITS de kullanılsa ilk adımın aynı olduğunu vurguladı. Kullanıcı bir arama kelimesi girdiğinde sorgu, TF-IDF skoruna dayalı olarak ters dizin (inverted index) tablosunun anahtar bölümündeki kelimelerle karşılaştırılır. Eşleşen girdilerin değer (belge) bölümündeki sayfalar alınır, birleştirilir (merge) ve TF-IDF skorlarına göre sıralanmış sonuçlar olarak sunulur. Yani anahtar kelimenin eşleştiği belgelerin TF-IDF ile bulunması adımı hem PageRank'te hem HITS'te vardır.

3.2 Kök kümeden taban kümeye

HITS, PageRank'ten farklı olarak sıralamayı sorguya bağlı yapar. Arama sonuçları bulunduktan sonra algoritma şöyle ilerler:

- TF-IDF'e göre sıralanmış bu arama sonuçlarına **kök küme (root set)** denir ve W ile gösterilir.
- Kök kümedeki sayfaların içeriği bilindiği için bu sayfalar bir adım ileri götürülür: Bu sayfaların bağlantı verdiği sayfalar bulunur.
- Aynı şekilde bu sayfalara bağlantı veren sayfalar da bulunur, yani bir adım geri gidilir.
- Bu sayfalar kök kümeye eklenerek genişletilmiş bir küme elde edilir. Buna **taban küme (base set)** denir.

Kök kümedeki sayfaların verdiği bağlantılar, sayfa içeriğindeki köprü bağlantılardan doğrudan okunur. Peki kök kümeye bağlantı veren sayfalar nasıl bulunur? Hoca burada tarama (crawling) aşamasını hatırlattı. Tarama sırasında indekslenen bütün sayfaların içeriği depolama alanına kaydedilmişti. Dolayısıyla webdeki bağlantı yapısı veritabanındaki veriden kolayca oluşturulabilir. Sayfaları yeniden taramaya gerek yoktur.

Taban küme elde edildikten sonra, PageRank'te olduğu gibi, bu küme üzerinden küçük bir web grafı kurulur. Bu grafla çalışabilmek için graf bir komşuluk matrisine (adjacency matrix) dönüştürülür. Komşuluk matrisi $n \times n$ boyutundadır. i düğümünden j düğümüne bir bağlantı varsa matristeki ilgili hücreye 1, yoksa 0 yazılır:

$$L_{ij} = \begin{cases} 1, & i \to j \text{ bağlantısı varsa} \\ 0, & \text{aksi halde} \end{cases}$$

HITS, PageRank gibi bütün web grafı üzerinde önceden bir değer hesaplamaz. Sorgu sırasında oluşturduğu bu bağlantı veri kümesi üzerinde hesap yapar. Hesapladığı iki değer vardır: otorite sıralaması (authority ranking) ve merkez sıralaması (hub ranking).

> **Ders sonrası not:** Hoca bir yerde, sorguyla eşleşen ilk URL'lere "base set" dedi. Doğrusu şudur: Sorguyla eşleşen ilk küme kök kümedir (root set). Bir adım ileri ve bir adım geri genişletilmiş küme taban kümedir (base set). Hoca anlatımın geri kalanında bu ayrımı doğru kullandı.

4. Otorite ve Merkez (Hub) Kavramları

Otorite (authority): Bir sayfa çok fazla bağlantı alıyorsa, bu o sayfanın belirli bir konuda otoriter bir sayfa olduğunu gösterir. Belirli bir konuda otoriter olduğu için çok bağlantı alır.

Merkez (hub): Bir sayfa çok sayıda sayfaya bağlantı veriyorsa, bu sayfa bir düzenleyici (organizer) görevi görür. Hocanın örnekleri Yahoo dizini (directory) ve Open Directory'ydi. Bu tür sayfalar çok sayıda bağlantıyı insanların belirli bir sınıflandırma (taxonomy) içinde görebilmesini sağlar.

İki kavram birbirine bağlıdır:

- Bir sayfanın iyi bir otorite olabilmesi için çok sayıda **iyi merkez** sayfasından bağlantı alması gerekir.
- Bir sayfanın iyi bir merkez olabilmesi için çok sayıda **iyi otorite** sayfasına bağlantı vermesi gerekir.

Yani iyi otoriteler iyi merkezlerden bağlantı alır, iyi merkezler de iyi otoritelere bağlantı verir. Bu yüzden otorite skoru ve merkez skoru **birbirini besleyen** değerlerdir.

5. HITS Denklemleri ve Matris Gösterimi

5.1 Tek bir düğüm için denklemler

Hoca, PageRank'te bir düğümün değerinin ona bağlantı veren düğümlerin oransal PageRank değerlerinin toplamı olduğunu hatırlattı. HITS'te de otorite ve merkez için birer denklem vardır:

- Bir sayfanın otorite skoru, o sayfaya bağlantı veren sayfaların **merkez skorlarının toplamıdır**.
- Bir sayfanın merkez skoru, o sayfanın bağlantı verdiği sayfaların **otorite skorlarının toplamıdır**.

$$a(i) = \sum_{(j,i)\in E} h(j), \qquad h(i) = \sum_{(i,j)\in E} a(j)$$

Burada E taban kümeden kurulan graftaki bağlantılar kümesidir.

Soru: O zaman her sayfanın iki skoru mu vardır? **Cevap:** Evet. HITS'te PageRank'teki gibi tek bir skor yoktur. Her sayfa için hem otorite hem merkez skoru hesaplanır. (Bu soruyu hoca öğrencinin ağzından kendisi sordu.)

5.2 Matris gösterimi

PageRank'te olduğu gibi, tek düğüm denklemleri küçük graflar için kullanılabilir. Büyük matrisler için denklemlerin matris gösterimiyle yazılması gerekir. Artık a bütün düğümlerin otorite skorlarını tutan bir sütun vektörüdür. Hoca slaytta bunu bir satır vektörünün transpozesi olarak yazdığını, ama aslında sütun vektörü olduğunu belirtti. h de bütün düğümlerin merkez skorlarını tutan bir sütun vektörüdür.

$$a = (a(1), a(2), \dots, a(n))^T, \qquad h = (h(1), h(2), \dots, h(n))^T$$

$$a = L^T h, \qquad h = L\, a$$

İlk denklemde $n \times n$ boyutlu L^T matrisi, $n \times 1$ boyutlu merkez vektörüyle çarpılır ve bütün düğümlerin otorite vektörü elde edilir. İkinci denklemde $n \times n$ boyutlu L matrisi $n \times 1$ boyutlu otorite vektörüyle çarpılır ve $n \times 1$ boyutlu merkez vektörü elde edilir.

Hoca, komşuluk matrisinin nereden geldiğini bir kez daha özetledi: Sorgu ters dizinde aranır, eşleşen URL'ler ayrılır (kök küme), bir bağlantı ileri ve bir bağlantı geri gidilerek bulunan sayfalar eklenir (taban küme), komşuluk matrisi bu taban kümedeki bütün bağlantılar için kurulur. Otorite ve merkez skorları da yalnız bu bağlantılar için hesaplanır. Yani HITS bütün web grafı için değil, yalnız sorgudan türetilen taban küme için hesaplanır.

6. Kuvvet Yinelemesi ile Çözüm ve Ortak Atıf İlişkisi

6.1 Yerine koyma

HITS de PageRank gibi kuvvet yinelemesi (power iteration) yöntemiyle hesaplanır. Bunun için denklemler birbirinin içine konur:

- Otorite denkleminde h yerine $h = L\, a$ yazılır: $a = L^T L\, a$.
- Merkez denkleminde a yerine $a = L^T h$ yazılır: $h = L\, L^T h$.

$$a = L^T L\, a, \qquad h = L\, L^T h$$

> **Ders sonrası not:** Hoca merkez denklemini sözlü anlatırken bir yerde "komşuluk matrisinin transpozesi çarpı hub vektörü" dedi. Doğrusu $h = L\, L^T h$'dir; yalnız $L^T h$ otorite vektörünü verir. Küçük bir test grafı üzerinde python ile yapılan kontrolde $h = L\, L^T h$ yinelemesinin merkez skorlarına, $L^T h$ yinelemesinin ise farklı bir vektöre yakınsadığı görüldü.

6.2 PageRank ile aynı mantık

$L^T L$, iki $n \times n$ matrisin çarpımı olduğu için yine $n \times n$ bir matristir. Hoca bunu tek bir matris gibi düşünmeyi önerdi. O zaman denklem, PageRank hesabında görülen yapının aynısıdır:

- Vektörün bir önceki durumu bu matrisle çarpılır ve vektörün bir sonraki andaki değeri bulunur.
- Bulunan değer tekrar yerine konur ve aynı matrisle yeniden çarpılır.
- Bu işlem, iki ardışık durum arasındaki fark çok küçülünceye kadar sürer. Yani vektör değişmeyen, durağan (stationary) bir vektöre yaklaşana kadar devam edilir.

Aynı işlem merkez skoru için de yapılır. Hem otorite hem merkez skorları için başlangıçta bir ilk değerden başlanır. Kuvvet yinelemesi uygulandıktan sonra taban kümedeki bütün düğümler için hem otorite hem merkez skoru bulunur. Hoca slaytta bu işlemin sözde kodunu (pseudo code) gösterdi: Algoritma taban kümedeki bağlantılardan oluşan grafı girdi olarak alır, bütün düğümlerin otorite ve merkez skorlarını çıktı olarak verir.

> **Ders sonrası not:** Derste her yinelemeden sonra a ve h vektörlerinin normalleştirildiğinden söz edilmedi. Normalleştirme yapılmazsa değerler her adımda büyür. Python ile küçük bir test grafında normalleştirmesiz 10 adımda değerlerin yüz binlere çıktığı, normalleştirmeli yinelemenin ise $L^T L$ ve $L\, L^T$ matrislerinin baskın özvektörlerine yakınsadığı doğrulandı.

6.3 Ortak atıf ve bibliyografik eşleşme ile ilişki

Hoca, bir önceki derste görülen ortak atıf ve bibliyografik eşleşme metrikleriyle bağlantıyı gösterdi:

- Otorite denklemindeki $L^T L$ ifadesi **ortak atıf matrisidir**.
- Merkez denklemindeki $L\,L^T$ ifadesi **bibliyografik eşleşme matrisidir**.

  Ek bilgi (derste anlatılmadı): $L^T L$ matrisinin (i, j) hücresi, hem i'ye hem j'ye bağlantı veren sayfaların sayısıdır; bu, ortak atıf tanımıyla aynıdır. $L\,L^T$ matrisinin (i, j) hücresi ise i ve j'nin birlikte bağlantı verdiği sayfaların sayısıdır; bu da bibliyografik eşleşmedir. Bu yüzden iyi otoriteler çok sayıda ortak atıf alan, iyi merkezler ise çok sayıda ortak kaynak paylaşan sayfalardır.

7. HITS'in Avantajları ve Dezavantajları

7.1 Avantaj: sorguya bağlı sıralama

HITS'in en büyük avantajı, arama sonuçlarını sorguya bağlı olarak sıralayabilmesidir. Hoca PageRank ile karşılaşılabilecek durumu ikinci bir örnekle anlattı. Wikipedia'da İkinci Dünya Savaşı'nı anlatan bir sayfa olsun. Oldukça otoriter bir sayfadır. Doğru bilgiyi yansıtması için çok sayıda farklı kişi tarafından izlenir ve düzenlenir. İkinci Dünya Savaşı ile ilgili bir arama yapıldığında bu sayfanın ilk 10 sonuçta görünmesi istenir. Ama İkinci Dünya Savaşı kelimelerini içeren, PageRank değeri bu Wikipedia sayfasından çok daha yüksek başka sayfalar varsa, PageRank bu sayfaları yalnız PageRank değerleri yüksek olduğu ve kelimeleri içerdiği için ilk 10'a getirir. Otoriter sayfa, PageRank değeri düşükse ilk 10'a giremez. Bu PageRank'e getirilen bir eleştiridir ve aynı zamanda HITS'in avantajıdır. HITS bir sorguya karşılık gelen sayfalar içinde en otoriter olanları bulup gösterir.

7.2 Dezavantaj 1: manipülasyon

HITS merkez skorunu hesaplarken bir sayfanın dış dünyaya verdiği bağlantılara bakar. Ama bir sayfanın verdiği bağlantıları o sayfanın yazarı belirler. HITS'i yanıltmak isteyen biri şunu yapabilir:

- Sayfasına iyi otoriter sayfalara giden bağlantılar koyarak sayfasının merkez skorunu yükseltir.
- Bunun yanında, iyi olmayan, otoriter olmayan sayfalara da bağlantılar verir.

HITS "iyi merkez sayfası iyi otoritelere bağlantı verir" varsayımına dayanır. Bu örnekte sayfa yalnız iyi sayfalara bağlantı vermediği için algoritma yanılabilir ve otoriter olmayan sayfaların otorite skoru yapay olarak yükselir.

7.3 Dezavantaj 2: konu kayması (topic drift)

Kök küme sorguyla eşleşen sayfalardan oluşur. Ama bir bağlantı ileri ve bir bağlantı geri gidildiğinde eklenen sayfalar sorgunun konusundan tamamen farklı olabilir. Bu durumda bağlantı analizi, ilk girilen konudan farklılaşan bir konu grubu üzerinde yapılmış olur. Buna **konu kayması (topic drift)** denir. HITS, sorguyla eşleşen sayfaların dışındaki sayfaları da analize kattığı için konu kayması oluşturabilir.

7.4 Dezavantaj 3: verimlilik

Hocaya göre en önemli dezavantaj, otorite ve merkez skorlarının **sorgu anında** hesaplanmasıdır. Sorgu geldiğinde ters dizinden sonuçların getirilmesi çok zaman almaz; bunu PageRank de yapar. Ters dizin bir karma tablo (hash table) gibi düşünülebilir ve bir anahtar kelimenin hangi girdiye karşılık geldiğini bulmanın karmaşıklığı çok düşüktür. Buna karşılık bir matris üzerinde kuvvet yinelemesiyle otorite ve merkez skorlarını hesaplamak zaman alır. Matris, bütün web kadar büyük değildir. Hoca bir sorguya karşılık gelen bağlantı sayısını 1000 ya da 2000 olarak düşünmeyi önerdi. Yine de bunun üzerinde matris kurup skor hesaplamak zaman gerektirir. PageRank'te skorlar önceden hesaplanmıştır ve sorgu anında yalnız bu skorlara göre yeniden sıralama yapılır. Bu yüzden HITS, PageRank'e göre daha az verimlidir.

> **Hocanın vurgusu:** Bağlantı analizi bölümünde sosyal ağ analizi, merkezilik ve prestij metrikleri, ortak atıf ve bibliyografik eşleşme, PageRank ve HITS konuşuldu. Bunların hepsinden sınavda sorumlusunuz. Oldukça önemli bir bölümdür; bu bölüme iyi çalışarak final sınavına gelin.

8. Web Kullanım Madenciliğine Giriş

Hoca ardından kitabın 12. bölümüne, web kullanım madenciliğine geçti. Dersin başında web madenciliğinin alt kolları konuşulmuştu:

- **Web içerik madenciliği (web content mining):** Verilen bir sorgunun web sayfalarının içerikleriyle nasıl eşleştirileceği ve sorguya karşılık gelen içeriklerin nasıl bulunacağı.
- **Web yapı madenciliği (web structure mining):** Bağlantı analizi yaparak bir sorguya karşılık gelen sayfalar içinde en popüler olanların bulunması.
- **Web kullanım madenciliği (web usage mining):** Bu derste konuşulan üçüncü kol.

  Ek bilgi (derste anlatılmadı): Bölüm numaraları Bing Liu'nun *Web Data Mining* kitabıyla uyumludur. Bu kitabın 12. bölümü (Web Usage Mining) büyük olasılıkla Bamshad Mobasher tarafından yazılmıştır.

Web siteleri, kullanıcıların oluşturduğu tıklama akışı (clickstream) verisini uygulama sunucusu tarafında kayıt (log) dosyalarında tutar. Hocaya göre iki tür kayıt dosyası olabilir:

- **Uygulama sunucusunun kayıt dosyası:** IIS, Tomcat, GlassFish gibi farklı türdeki uygulama sunucularının kendi tuttuğu kayıt dosyası.
- **Uygulamanın kayıt dosyası:** Bu sunucu üzerinde çalışan kendi uygulamamızın tuttuğu kayıt dosyası.

Birden fazla kayıt dosyası olabilir. Ama hepsi sonuçta kullanıcının sayfa içinde yaptığı bütün etkinlikleri sunucu tarafında kayıt altına alır.

Web kullanım madenciliğinin amacı, bir kişinin ya da çok sayıda insanın bir siteyi incelerken oluşturduğu örüntüleri (pattern) bulmaktır. Çok sayıda insan bir siteyi kullanırken kolektif olarak belli davranışlar gösteriyorsa ve bu davranış biçimi yakalanabilirse, bunun üzerine aksiyonlar alınabilir. Hocanın örnekleri e-ticaret sitesi üzerindendi:

- Kampanyalar oluşturmak.
- Birlikte satılan ürünleri bulmak.
- Siteye gelenlerin sırayla hangi etkinlikleri yaptığını bulmak.
- Siteye gelen kişinin ürün almadan çıkıp gideceğini mi, yoksa ürün almak amacıyla mı geldiğini sınıflandırmak, tahmin etmek.

Yani kayıt dosyaları analiz edilerek davranış örüntüleri yakalanmaya çalışılır.

9. Veri Kaynakları ve Tıklama Akışı Verisi

9.1 Veri kaynakları

Web kullanım madenciliğinde üzerinde çalışılan veriler şunlardır:

- Web sunucusu kayıt dosyaları.
- Uygulama verisi, yani sunucu altında çalışan kendi web uygulamamızın ürettiği veri.
- Sunucu altına konan sitenin bağlantı yapısı.
- Sitenin arka planda etkileşimde olduğu eski (legacy) veritabanları. Örneğin müşteri ilişkileri yönetimi (CRM) veritabanı ya da müşterilerle ilgili verileri tutan başka veritabanları.

Bu kaynakların hepsi her zaman bulunmayabilir. Bulunanlar mutlaka bütünleştirilmeli ve tek bir işlem veritabanına (transaction database) dönüştürülmelidir. Ancak o zaman bunun üzerinde örüntü var mı diye bakılabilir ya da bir model kurulup tahmin yapılabilir.

Bu veriler arasında en fazla yer tutanı web sunucusu kayıt dosyalarıdır. Bu dosyalar sitede gerçekleşen bütün tıklamaları kaydettiği için, çok etkileşim olan bir sitede zamanla çok büyük kayıt dosyaları oluşur. Bu yüzden web kullanım madenciliğinde ağırlıklı olarak tıklama akışı verisi üzerinde çalışılır.

9.2 Bir kayıt satırından neler öğrenilir?

Hoca slaytta bir uygulama sunucusu kayıt dosyası örneği gösterdi. Her satır bir HTTP isteğidir (request). Böyle bir satırdan şunlar öğrenilebilir:

- İsteğin geldiği tarih ve saat.

- İsteğin geldiği sunucu, yani kayıt dosyasının bulunduğu sunucu.
- Bu sunucu altında hangi sayfanın istendiği. Örnekte `classes` dizini altında `cs589` altındaki `papers.html` dosyası istenmişti.
- Yönlendiren sayfa (referrer): İsteği gönderen kişinin hangi sayfadaki bir bağlantıya tıklayarak bu sayfaya geldiği.
- Kullanıcının işletim sistemi ve sürümü, kullandığı tarayıcı (browser) ve tarayıcının sürümü.
- Kullanıcının IP adresi. Bu adres statik ya da dinamik olabilir.

Hoca örnekte görünmese de iki bilgi daha olabileceğini söyledi:

- **Kullanıcı kimliği (user ID):** Tarayıcıda çerezleri engelleme seçeneği seçilmemişse kullanıcı tarafına bir çerez (cookie) konabilir. Kullanıcı için bir kullanıcı kimliği oluşturulup çerezin içine yazılır. Kullanıcı siteye tekrar geldiğinde bu kimlikle izlenebilir.
- **Oturum kimliği (session ID):** Kullanıcı her geldiğinde onun için bir oturum kimliği yaratılır. Kullanıcının o kısa zaman dilimindeki hareketleri bu kimlikle izlenir.

Gösterilen örnekte kullanıcı kimliği ve oturum kimliği bilgileri yoktu.

10. Kullanıcı İşlem Veritabanı ve Veri Hazırlama Süreci

10.1 Kullanıcı işlem veritabanı

Bir kayıt dosyasından örüntü çıkarmak, sınıflandırıcı model kurmak ya da kümeleme yapmak için önce bu verinin bir **kullanıcı işlem veritabanına (user transaction database)** dönüştürülmesi gerekir. Hoca bunu birliktelik kuralı madenciliği (association rule mining) ve sıralı örüntü madenciliği (sequential pattern mining) konularında görülen işlem veritabanına benzetti.

Bu veritabanında her satır bir kullanıcıya aittir. Bir sütunda kullanıcının kimliği, diğer sütunda kullanıcının gerçekleştirdiği etkinlikler tutulur. Etkinlikler sıralı olarak ve oturum bazında görülür. Bu veritabanı web uygulaması ve sunucu kayıt dosyalarından oluşturulur.

10.2 Genel adımlar

Hoca önce süreci genel olarak anlattı:

- **Veri temizleme (data cleaning):** İşe yaramayacak HTTP istekleri çıkarılır. Örneğin siteyi yalnız kullanıcılar ziyaret etmez; Googlebot, MSNbot gibi siteyi indekslemeye çalışan arama motoru tarayıcıları (crawler, spider) da HTTP isteği yapar. Bunların verisi kayıtlardan temizlenmelidir.
- **Sayfa görüntüleme tanımlama (pageview identification):** Bir sayfada resimler, müzik dosyaları gibi çok sayıda çoklu ortam nesnesi vardır. Kullanıcı tek bir sayfa ister, ama tarayıcı arka planda her nesne için ayrı HTTP isteği gönderir. Sunucu tarafında sanki çok sayıda istek yapılmış gibi kayıt dosyasına girdiler düşer. Tek bir sayfa isteği için 10 girdi oluşmuşsa, bu 10 girdinin aslında tek bir sayfa görüntülemesine karşılık geldiği belirlenmelidir.
- **Oturum tanımlama:** Bir kullanıcı sabah gelip bir şeyler yapar, akşam gelip başka şeyler yapar. Bu iki ayrı oturumdur ve bunların tespit edilmesi gerekir.
- **Veri bütünleştirme (data integration):** Sunucu kayıt dosyalarından, uygulama kayıt dosyalarından ve sitenin bağlantı yapısından gelen veriler bütünleştirilerek kullanıcı işlem veritabanı oluşturulur.
- **Veri dönüştürme (data transformation):** Veri bir veri tipinden diğerine dönüştürülür ve kullanıcı bazlı bir veritabanı kurulur.

Bunların hepsi veri ön işleme (data preprocessing) çalışmasıdır. Kullanıcı işlem veritabanı oluşturulduktan sonra dersin başında görülen teknikler uygulanır: birliktelik kuralı madenciliği, sıralı örüntü madenciliği, kümeleme (işlem tabanlı ya da sayfa görüntüleme tabanlı), birbiriyle ilişkili ürünleri bulmak için korelasyon analizi ve müşterilerin terk etme ya da satın alma davranışını tahmin eden modeller. Bu teknikler sonucunda modeller ve örüntüler elde edilir. Bunlar üzerinde filtreleme yapılır, modeller birleştirilir, yeni modeller kurulur, birleştirme (aggregation) yapılır. Sonuçta müşterilerin davranışını anlamaya yarayan, anlam çıkarılabilir modeller oluşturulur.

10.3 Veri hazırlamanın ayrıntılı sırası

Hoca bir sonraki slaytta veri hazırlama kısmını daha ayrıntılı gösterdi:

- **Veri birleştirme (data fusion):** Sunucu ya da uygulama kayıt dosyaları gibi birden fazla kaynaktan gelen veri birleştirilir.
- **Veri temizleme:** Tarayıcıların bıraktığı HTTP istekleri çıkarılır.
- **Sayfa görüntüleme tanımlama:** Tek bir bağlantı isteği için tarayıcının yaptığı çok sayıda isteğin tek bir sayfa görüntülemesine ait olduğu belirlenir.
- **Oturum oluşturma (sessionization):** Oturumlar tespit edilir.
- **Bölüm tanımlama (episode identification):** Kullanıcının bir oturum içinde gerçekleştirmek istediği "hikaye" bulunur. Bu bir satın alma hikayesi mi, sepete ürün koyma hikayesi mi, favori listesine ürün ekleme hikayesi mi?

Bu işlemler sonucunda ön işlenmiş bir tıklama akışı verisi, yani bir ara veritabanı elde edilir. Ardından bu ara veritabanı, arka planda müşterilerle ilgili tutulan CRM gibi diğer operasyonel veritabanlarından gelen veriyle bütünleştirilir ve zenginleştirilir. Site içeriğinden gelen veriyle de zenginleştirilir. Sonra kullanıcı işlem veritabanına dönüştürülür.

11. Bölüm (Episode) Tanımlama

Aradan sonra hoca bölüm tanımlamayı açtı. Bir kullanıcının hangi oturumları oluşturduğu tespit edildikten sonra, oturum içindeki hareketlerin **ne amaçla** yapıldığının bulunması gerekir. Bir e-ticaret sitesinde farklı davranışlar olabilir:

- Satın alma.
- Ürün iade etme.
- Yardım arama.
- Bir ürünü arama.
- Favorilere ekleme.

Kullanıcı bu davranışları gerçekleştirmek için çok sayıda etkinlik yapar. Örneğin ürünün sayfasına girer, görüntüler, fiyata bakar, yorumlara bakar, ürünü sepete koyar, başka işlemler yapar, sonra sepete dönüp kredi kartıyla ödeme bölümüne gider. Yapılması gereken, bu etkinlikler arasından kullanıcının o oturumu (30 dakika ya da 60 dakika ne kadarsa) sonuçta ne yapmak için oluşturduğunu bulmaktır. Bölüm tanımlama budur.

Bölüm tanımlama iki yolla yapılabilir:

- **Kural tabanlı:** "Bir kullanıcı sitemde şu sayfalara birbirini takip eden şu sırada girmişse, şu bölümü gerçekleştirmek istiyordur." Örneğin "bir ürünü favorilerime ekle" bölümü.
- **Sınıflandırma:** Bir sınıflandırıcı algoritmayla model kurulur. Model, kişinin yaptığı sayfa görüntülemelerinden ve bunların sırasından yola çıkarak bir etiket atar. Örneğin "bu kişi satın alma davranışı gösteriyor" ya da "satın almadan sistemi terk etme davranışı gösteriyor".

Hocaya göre elimizdeki veriyle yapabiliyorsak bölüm tanımlamayı mutlaka yapmalıyız.

12. Veri Temizleme

Hoca veri temizlemede dikkat edilecek noktaları sıraladı:

- **Konuyla ilgisiz kayıtlar:** Hedefimiz örneğin kullanıcıların sitedeki satın alma davranışını etiketlemekse, bunun dışında kalan, bizimle ilgili olmayan kayıt girdileri kullanılmaz.
- **Tarayıcı kayıtları:** Arama motoru tarayıcılarının bıraktığı girdiler kullanılmaz.
- **Hatalı istekler:** Kayıt dosyasına hatalı biçimde düşmüş istekler tutulmaz.

Bunun yanında vekil önbellekler (proxy cache) nedeniyle kayıt dosyasına düşmemiş eksik etkinlikler, eksik tıklama akışları varsa bunlar tamamlanır (bkz. Bölüm 15.2).

13. Kullanıcı Tanımlama

13.1 Neden gerekli?

Hoca burada öğrencilerin sorabileceği iki soruyu kendisi sordu ve cevapladı.

Soru: Kullanıcı kimliğini zaten çerezden almıyor muyduk? Kullanıcıyı tespit etmek için neden uğraşıyoruz? **Cevap:** Her zaman böyle olmuyor. Çoğu kişi tarayıcısında çerezleri engelliyor. Bu durumda gelen kişinin kullanıcı kimliği izlenemez; kişinin daha önce gelmiş biri olduğunu anlamak için tekil bir anahtar kullanılamaz.

Soru: Siteler kişileri oturum açmaya (login) zorluyor. Oturum açınca kişi kimlik doğrulama jetonunu (authentication token) gönderdiği için kim olduğu izlenemez mi? **Cevap:** Burada haklısınız; siteye oturum açmışsa kişi oturum süresince izlenebilir. Ama çoğu site, özellikle e-ticaret siteleri, kullanıcıları bir ürün satın alıncaya kadar oturum açmaya zorlamaz. Hatta oturum açmadan, yalnız kredi kartı bilgilerini girerek ürün satın alınabilir; sitenin üyesi olmayı gerektirmeyen siteler vardır.

Bu nedenle kullanıcının kim olduğunu, daha önce gelmiş bir kullanıcı olup olmadığını tespit etmek için **kullanıcı tanımlama (user identification)** yapılmalıdır. Sonra kullanıcının sabah, öğlen, akşam gelişlerinde oluşan tıklama akışı verisinden hangi oturumları oluşturduğu bulunur. Bir oturumda birçok sayfa görüntülenir ve her görüntüleme çok sayıda istek üretir; bu yüzden oturumdaki sayfa görüntüleme sayısı da tespit edilmelidir. Vekil önbellek nedeniyle eksik kalan yollar tamamlanırsa sayfa görüntüleme tanımlama da daha sağlıklı yapılır. Bütün bunlardan sonra oturumun hangi kullanıcıya ait olduğu ve içinde hangi sayfaların görüntülendiği bilinir; son adım bölüm tanımlamadır.

13.2 Çerez ile ve IP + ajan ile tanımlama

- **Çerez:** Tarayıcı çerez kullanımına izin veriyorsa, site her kullanıcının tarayıcısına bir çerez koyar. Kullanıcının tarayıcısı siteye tekrar geldiğinde HTTP isteklerini bu çerezdeki tekil kullanıcı kimliğiyle gönderir. Böylece tekrar gelişlerde kullanıcının daha önce gelmiş biri olduğu anlaşılır.
- **IP + kullanıcı ajanı (user agent):** Çereze izin verilmemişse kullanıcının IP adresi, işletim sistemi ve sürümü, tarayıcısı ve sürümü birbirine eklenir (append) ve tek bir tekil ifadeye dönüştürülür. Bu ifade kimlik (identifier) olarak kullanılır.

IP + ajan yanıltıcı bir kimliktir; %100 doğru değildir. Kullanıcı ortak bir ortamdaysa, örneğin bir kafede ya da paylaşılan bir ağdaysa, sürekli dinamik IP alır. Siteye gelen IP adresi sürekli değiştiği için kişinin daha önce gelip gelmediğini izlemek güçleşir. Ama kişi evinden, statik bir IP'den ve tek bir bilgisayardan bağlanıyorsa bu tekil kimlikle izlenebilir.

13.3 Yöntemlerin karşılaştırılması

Hoca slayttaki tabloyla yöntemlerin avantaj ve dezavantajlarını karşılaştırdı:

Yöntem	Avantaj	Dezavantaj
Çerez (cookie)	Tekrar tekrar gelen kullanıcıyı izlemek için çok avantajlı	Tarayıcılar çoğu zaman çerezi reddeder; kullanıcılar tarayıcıyı çerezleri engelleyecek şekilde ayarlar
Oturum açma (login, registration)	Kullanıcı hangi cihaz ve tarayıcıdan (mobil, Edge, Firefox, Chrome) gelirse gelsin izlenir	Bütün kullanıcılar oturum açarak gezinmez
Yazılım ajanı (software agent), ör. sitenin mobil uygulaması	Kullanıcı rahatça izlenir	Kullanıcılar her sitenin uygulamasını indirmek istemeyebilir, doğrudan tarayıcıyı kullanmak isteyebilir

Yöntem	Avantaj	Dezavantaj
Gömülü oturum kimliği (embedded session ID)	Tarayıcının engellemesi kolay değil	Kullanıcının yalnız o oturumdaki hareketleri izlenir; farklı zamanlardaki oturumlar ilişkilendirilemez
IP + ajan	Ek bilgi gerektirmez	%100 garantisi yok; dinamik IP'ler kullanıcının tanınmasını engeller

Hoca oturum açmanın dezavantajını gündelik bir örnekle açıkladı: Bir e-ticaret sitesinde hesabınız olsa bile siteyi çoğu zaman oturum açmadan gezersiniz.

13.4 Kullanıcı tanımlama örneği

Hoca slaytta farklı kullanıcıların verilerinin karıştığı bir kayıt örneği gösterdi ve sordu: Burada hangi kullanıcı tanımlama yöntemi kullanılmalı? Kayıtta ne çerezdeki kullanıcı kimliği, ne oturum kimliği, ne de giriş bilgisi vardı. Bu yüzden kullanılabilecek tek yöntem IP + ajandır.

- Birinci kullanıcı: Bir IP adresi ile Internet Explorer 5 ve Windows 2000 ajan bilgisi birleştirilerek tekil bir kimlik oluşturuldu. Bu kimliğe uyan bütün satırlar, kaydın farklı yerlerinde olsalar da, birinci kullanıcının verisi olarak bir araya toplandı.
- İkinci kullanıcı: Başka bir IP adresi ile Internet Explorer 6, Windows XP ve ajan dizgisindeki ek yazılım paketi ve güncelleme bilgisi birleştirildi.
- Üçüncü kullanıcı da aynı biçimde tanımlandı.

14. Oturum Tanımlama ve Sezgisel Yöntemler

14.1 Gömülü oturum kimliği

Oturum, gelen HTTP isteğindeki gömülü oturum kimliğiyle ayırt edilebilir. Örneğin kullanıcı oturum açar, site ona bir oturum kimliği verir ve kullanıcı bundan sonraki her isteğini bu kimlikle gönderir. Bu durumda kayıt dosyası analiz edilirken gerçek oturumlar yakalanır. Bu en iyi durumdur.

Gömülü oturum kimliği yoksa oturumlar **sezgisel yöntemlerle (heuristics)** tespit edilir. Bu sırada kullanıcının kim olduğu kesin biliniyor olabilir ya da IP + ajanla %100 olmasa da yaklaşık olarak bulunmuş olabilir.

14.2 Zamana dayalı sezgiler

Hoca, web sayfalarını ziyaret edenlerin davranışlarını inceleyen çalışmaların ortalamada bir oturumun yaklaşık 30 dakika sürdüğünü ortaya koyduğunu söyledi. Bu değer azalıp çoğalabilir ve yapılandırılabilir (configurable) bir değerdir; siteden siteye değişir. E-ticaret sitesinde daha kısa olabilir. Metnin çok, resmin az olduğu, insanların daha fazla zaman harcadığı sitelerde daha uzun olabilir.

İki zaman kuralı kullanılır:

- **Toplam oturum süresi:** Kullanıcının siteye yaptığı ilk istekten başlayarak 30 dakikalık zaman dilimindeki bütün davranışları bir oturum sayılır.
- **Sayfada kalma süresi:** Oturum içinde bir sayfada en fazla 10 dakika kalınabilir. Kullanıcı bir sayfada 10 dakikadan fazla kalmışsa oturum orada bitirilir. Oturum için 30 dakika belirlenmiş olsa bile 11. dakikaya geçildiğinde oturum kesilir. Kullanıcının bir sonraki hareketi nerede başlıyorsa yeni oturum oradan başlatılır.

Bu yaklaşım %100 doğru olmayabilir, ama gömülü oturum kimliği yokken oturumun nerede başlayıp nerede bittiğine dair bir yaklaşım ortaya koymak gerekir.

> **Ders sonrası not:** Hoca 30 dakikalık oturum sınırı ile sayfa başına 10 dakika sınırından "en fazla, yani en az 3 sayfa" gibi bir sonuç çıkarmaya çalıştı, sonra kendini

düzeltti. İki sınır birbirinden bağımsızdır; sayfa sayısı için bir alt sınır yoktur. 10 dakika kuralı yalnız iki ardışık istek arasındaki sürenin sınırıdır.

14.3 Zamana dayalı oturum oluşturma örneği

Hoca bir kullanıcıya ait istek tablosu gösterdi. İsteklerden zaman, IP adresi, istenen URL ve yönlendiren adres alınmıştı. Kullanıcı tanımlama zaten yapılmıştı; yani bütün satırların aynı kullanıcıya ait olduğu biliniyordu. Soru: Burada kaç oturum var?

- **Verilen:** Oturum süresi 30 dakika. İlk istek 0:01'de.
- **Adım 1:** İlk isteğin zamanına 30 dakika eklenir: $0{:}01 + 30$ dk $= 0{:}31$. 0:01 ile 0:31 arasına giren bütün istekler 1. oturumdur.
- **Adım 2:** Bu aralıktan sonraki ilk istek 1:15'te gelmiştir. $1{:}15 > 0{:}31$ olduğu için bu istek yeni oturumu başlatır.
- **Adım 3:** 1:15'ten başlayan 30 dakikalık aralıktaki ($1{:}15$–$1{:}45$) bütün istekler 2. oturumdur.

14.4 Gezinmeye dayalı sezgi

Hoca bir de gezinmeye (navigation) dayalı sezgisel yaklaşımı anlattı. HTTP isteğinde kullanıcının hangi sayfadan geldiğini belirten bir yönlendiren (referrer) alanı vardır. İlk gelen istekte bu alan boşsa, kullanıcı tarayıcının adres çubuğuna gitmek istediği yeri kendisi yazmış demektir; bir yönlendiren sayfa yoktur. Bu durum yakalanırsa burası oturumun başlangıcı sayılır. Bu başlangıçtan itibaren bir sonraki yönlendirensiz isteğe kadar yapılan etkinlikler aynı oturumdaki etkinlikler kabul edilir.

> **Ders sonrası not:** Hoca bu alanı bir yerde "HREF denilen alan" diye andı, hemen ardından "referral alan" dedi. Doğrusu HTTP isteğindeki `Referer` başlığıdır. Ayrıca kitaptaki yönlendiren tabanlı sezgi büyük olasılıkla şöyle tanımlanır: Bir isteğin yönlendireni o anki oturumda daha önce görüntülenmiş bir sayfaysa istek o oturuma eklenir, değilse yeni oturum başlar. Hocanın anlattığı "yönlendiren boşsa yeni oturum" kuralı bunun özel bir durumudur.

15. Sayfa Görüntüleme Tanımlama ve Yol Tamamlama

15.1 Sayfa görüntüleme tanımlama

Kullanıcı tarayıcıya bir URL girdiğinde tarayıcı o sayfayı getirmek için birçok farklı kaynağa HTTP isteği yapabilir, çünkü sayfanın içindeki web nesnelerini ayrı ayrı indirmesi gerekir. Bu isteklerin hepsinin tek bir sayfaya ait olduğunu tespit etmek için sayfa görüntüleme tanımlama yapılır.

15.2 Yol tamamlama (path completion)

Çoğu sitenin önünde bir vekil sunucu (proxy server) bulunur. Kullanıcı siteyi gezerken daha önce istediği bir sayfaya geri dönerse, istek sunucuya gönderilmeden önbellekten karşılanır. Bu durumda kullanıcının sunucu tarafındaki hareketleri analiz edilirken bazı isteklerin eksik olduğu görülür. Eksik kalan bu yolların tamamlanması gerekir.

Hocanın örneği:

- **Kullanıcının gerçek gezinmesi:** A sayfasına doğrudan URL yazarak geldi (yönlendiren yok). A'dan B'ye, B'den D'ye, D'den E'ye gitti. E'den tekrar D'ye, D'den tekrar B'ye döndü. Sonra B'den C'ye gitti.
- **Sunucu kaydı:** D ve B daha önce istendiği için vekil önbellekte vardı. E'den D'ye ve D'den B'ye dönüşler önbellekten karşılandı, sunucuya gitmedi. C ise daha önce önbellekte olmadığı için B'den C'ye geçiş kayıtta görünür.

Kayıt: $A \to B \to D \to E \to C$

Tamamlanmış yol: $A \to B \to D \to E \to D \to B \to C$

Kayıtta E'den sonra doğrudan C geliyor. Ama C'ye giden isteğin yönlendireni B'dir ve E'den C'ye bir bağlantı yoktur (ara adım eklendi). Sitenin bağlantı yapısına bakılarak eksik istekler (E→D ve D→B) kayda eklenir. Bu tamamlama yapılmazsa kullanıcının sayfa üzerinde gerçekte ne yaptığı izlenemez.

> **Ders sonrası not:** Hoca örneği anlatırken önce "B'den C'ye gitmişim" dedi, hemen ardından "öncelikle B'den D'ye gitmişim" diye düzeltti. Doğru sıra yukarıdaki gibidir: C'ye en sonda, B'ye geri dönüldükten sonra gidilir.

16. Olaylar (Events) ve Tıklama Akışı

Şimdiye kadar konuşulan kayıt dosyalarında hep tıklama akışı verisi vardı. Tıklama akışı verisi tıklama verisidir: Bir sayfada bir bağlantıya, bir düğmeye tıklamak ya da bir onay kutusunu işaretlemek bir tıklama oluşturur. Her istek bir kayıt girdisi olarak yazılır.

Bunun yanında site üzerinde **olaylar (events)** tanımlanabilir. Olay, tıklama akışından farklıdır; birden fazla tıklamanın birleşiminden oluşabilir. Hocanın örnekleri:

- **Ürün görüntüleme (product view):** Bir ürünü görüntülemek, büyütmek; bunu yaparken birden fazla tıklama yapılır.
- **Ürün tıklama (product click-through):** Bir ürün hakkında bilgi almak istendiğinde gerçekleşen olay.
- **Sepet değişikliği (shopping cart change):** Sepete ürün eklemek ya da sepetten ürün çıkarmak.
- **Satın alma** ve **ürüne fiyat teklif etme**.

Tıklama akışı verisine anlam kazandırmak için sayfadaki bu olaylar tanımlanır ve tıklama akışı verisiyle eşleştirilir. Bunun yolu şudur: Sayfayı enstrümante eden, yani sayfayı hazırlayan kişi biz olduğumuz için, sayfada gerçekleşen bir etkinliği kayıt dosyasına bir olay kimliği (event ID) ile yazdırabiliriz. Okurken de her tıklama akışı girdisinin hangi olaya ait olduğu girdide yazılı olduğu için öğrenilir.

Bu yalnız uygulamanın kendi kayıt dosyası için yapılabilir. IIS ya da Apache Tomcat gibi bir sunucunun kayıt dosyasının bir sayfa içinde tanımlanan olayları izlemesi imkansızdır.

17. Örüntü Keşfi, Mimari ve Kullanım Alanları

17.1 Sürecin özeti

Hoca bütün süreci bir slaytta özetledi:

- Ham uygulama sunucusu verisi ya da uygulamanın kendi kayıt dosyaları alınır.
- Ön işleme yapılır: Birden fazla kayıt dosyası varsa bütünleştirilir, veri temizlenir, kullanıcı tanımlama, oturum tanımlama ve bölüm tanımlama yapılır.
- Elde edilen ön işlenmiş tıklama akışı verisine birliktelik kuralı madenciliği, sıralı örüntü madenciliği, sınıflandırma ya da kümeleme algoritmaları uygulanır.
- Bulunan örüntüler ve kurallar analiz edilir, kullanılabilir kurallar ve modeller seçilir.
- Ortaya çıkan ilginç bilgi ve çıkarımlar pazarlama biriminin ya da ilgili iş biriminin önüne konur.

17.2 Bölüm tanımlama sonucu örneği

Hoca, sürecin sonunda varılmak istenen şeyi bir örnekle gösterdi. Bir kayıt dosyasında HTTP GET yöntemiyle yapılmış çok sayıda arama isteği vardı. Bölüm tanımlamadan sonra kullanıcının davranışı şöyle okundu:

- Önce konuma (location) göre arama yaptı.
- Sonra elde edilen sonuçlar üzerinde hem konuma hem fiyata göre arama yaptı.
- Sonra bunlar arasından bir ürünü seçti.

Web kullanım madenciliğiyle hedeflenen budur: Kayıt dosyasındaki girdilerden kullanıcının davranışlarını sıralı, anlamlı ifadelere dönüştürmek.

17.3 Bir e-ticaret sitesi için mimari

Hoca bir e-ticaret sitesinin kayıt dosyaları üzerinde analiz yapan bir yazılımın mimarisini gösterdi:

- Site içeriği ve bu içeriği analiz eden bir modül.
- Sunucu kayıt dosyaları. Bunlar üzerinde veri temizleme, oturum oluşturma, kullanıcı tanımlama ve bölüm tanımlama yapılır, sonuç veri bütünleştirme modülüne gönderilir.
- Veri bütünleştirme modülü, bu veriyi CRM gibi müşteri ilişkileri veritabanlarından gelen müşteri bilgileriyle bütünleştirir. Böylece veri, üzerinde veri madenciliği yapılabilir hale gelir.
- Veri madenciliği tarafında, veri madenciliği dersinde görülen veri ambarları (data mart) kurulabilir ve bunlar üzerinde OLAP analizleri yapılabilir. Yine bu veri ambarı üzerinde çalışan veri madenciliği kütüphaneleriyle modeller kurulabilir.

17.4 Hedeflenen çıkarımlar

Web kullanım verisinden elde edilen kullanıcı işlem veritabanı üzerinde şunlar yapılabilir:

- **Birliktelik kuralları:** Birlikte satılan ürünler, daha doğrusu birliktelikler bulunur. Bunlardan kurallar çıkarılır: “Bu iki ürün birlikte satılıyor; birinci ürünü alan yüzde şu olasılıkla ikinci ürünü de alır.”
- **Sıralı örüntüler:** Sıralı satın almalar ya da sıralı aksiyonlar bulunur. “Bir kişi önce bu ürünü alırsa, şu olasılıkla ikinci ürünü de alır.” Bu yalnız ürünlerle sınırlı değildir: Bir sayfayı ziyaret eden kişinin sonra hangi sayfayı ziyaret edeceği, bir ürünü görüntüleyen kişinin sonra onu satın alıp almayacağı da bulunabilir.
- **Kümeleme:** Kullanıcılar sitedeki davranışlarına göre bölümlere ayrılır (segmentation). Kullanıcıların sitedeki istekleri kümelenerek ısı haritaları (heat map) oluşturulur. Ürünlerin ve sayfaların kategorilere ayrılmasıyla insanların sitede en çok hangi sayfalarda, hangi kategorilerde dolaştığı bulunur.
- **Sınıflandırma:** Müşterilerin terk etme davranışı tahmin edilir: Siteye girdikten sonra bir ürünü satın alacaklar mı, yoksa satın almadan terk edip gidecekler mi?

Hoca son olarak dönem boyunca öğrenilen tekniklerin hangi uygulamalarda kullanıldığını gösteren bir tabloyu özetledi: Markov zincirleri (PageRank’in nasıl çalıştığı bununla öğrenildi), sıralı örüntü madenciliği, birliktelik kuralı madenciliği, kümeleme (kullanıcı kümeleme ya da sayfa kümeleme için) ve sınıflandırma. Sınıflandırmayla örneğin bir sitede sahtecilik (fraud) olup olmadığı tespit edilebilir ya da gelen kullanıcıların ne tipte kullanıcılar olduğu tahmin edilebilir.

> **Hocanın vurgusu:** Web kullanım madenciliğinde özellikle oturum oluşturma, kullanıcı tanımlama, oturum tanımlama, eksik URL tamamlama (path completion) ve oturum oluşturma sezgileri (sessionization heuristics) önemli konulardır. Bunlara iyi çalışın; sınavda karşınıza çıkabilecek sorular arasında olacak.

Örnek Sorular

Bu derste hazır kare ve video kullanılmadı. Sorular, hocanın derste sözlü sorduğu ya da öğrencinin ağzından kendisinin sorup cevapladığı sorulardır.

Soru 1 (derste sözlü, 00:08:46): PageRank’in sorgudan bağımsız olmasına bir dezavantaj olarak nasıl bakılabilir?

> **Çözüm (derste verildi):** PageRank, sorguyla eşleşen sayfaları anahtar kelimeden bağımsız skorlarına göre yeniden sıralar. “Web data mining” aramasında en otoriter sayfa bu başlıklı bir kitabın sayfasıdır; ama PageRank değeri düşükse, kelimeleri içeren ve daha popüler sayfalar onu ilk 10 sonucun dışına iter.

Soru 2 (derste sözlü, 00:17:44): HITS’te taban küme kurulurken kök kümeye bağlantı veren sayfalar nasıl bulunur?

> **Çözüm (derste verildi):** Tarama sırasında indekslenen bütün sayfaların içeriği depolamaya kaydedilmiştir. Webdeki bağlantı yapısı bu veritabanından oluşturulur;

sayfaları yeniden taramaya gerek yoktur. Kök kümenin verdiği bağlantılar ise doğrudan sayfaların içeriğindeki köprü bağlantılardan okunur.

Soru 3 (derste sözlü, 00:21:42): HITS'te her sayfanın iki skoru mu vardır?

> **Çözüm (derste verildi):** Evet. PageRank'teki gibi tek skor yoktur. Her sayfa için otorite skoru $a(i) = \sum_{j \to i} h(j)$ ve merkez skoru $h(i) = \sum_{i \to j} a(j)$ hesaplanır.

Soru 4 (derste sözlü, 00:32:46): HITS'in dezavantajları nelerdir?

> **Çözüm (derste verildi):** (1) Merkez skoru, yazarın kendi verdiği bağlantılara dayandığı için manipülasyona açıktır. (2) Kök kümeden bir adım ileri ve geri gidilince eklenen sayfalar nedeniyle konu kayması (topic drift) olabilir. (3) Skorlar sorgu anında kuvvet yinelemesiyle hesaplandığı için PageRank'ten daha az verimlidir; hocaya göre en önemli dezavantaj budur.

Soru 5 (derste sözlü, 01:13:40): Kullanıcı kimliğini çerezden alabiliyorsak ya da kullanıcı oturum açıyorsa, kullanıcı tanımlamaya neden gerek var?

> **Çözüm (derste verildi):** Çoğu kullanıcı tarayıcısında çerezleri engeller. Çoğu e-ticaret sitesi de satın almaya kadar oturum açmayı zorunlu tutmaz; bazıları üye olmadan satın almaya izin verir. Bu yüzden kullanıcının daha önce gelip gelmediğini bulmak için kullanıcı tanımlama gerekir.

Soru 6 (derste sözlü, 01:32:15): Aynı kullanıcıya ait, ilki 0:01'de, oturum sınırından sonraki ilki 1:15'te gelen istekler var. Zamana dayalı sezgiyle (30 dakika) kaç oturum oluşur?

> **Çözüm (derste verildi):** 0:01'e 30 dakika eklenir; 0:31'e kadarki istekler 1. oturumdur. 1:15 > 0:31 olduğu için 1:15'teki istek 2. oturumu başlatır; 1:15–1:45 arasındaki istekler 2. oturumdur. Sonuç: 2 oturum.

Soru 7 (derste sözlü, 01:37:32): Kayıtta çerezdeki kullanıcı kimliği, oturum kimliği ve giriş bilgisi yoksa hangi kullanıcı tanımlama yöntemi kullanılmalıdır?

> **Çözüm (derste verildi):** IP + ajan. IP adresi ile işletim sistemi, sürümü, tarayıcı ve sürümü birleştirilerek tekil bir kimlik oluşturulur ve bu kimliğe uyan bütün satırlar aynı kullanıcıya atanır (ör. IE 5 / Windows 2000 ile gelen IP birinci kullanıcı, IE 6 / Windows XP ile gelen başka bir IP ikinci kullanıcı).

Soru 8 (derste sözlü, 01:41:41): Kullanıcı A → B → D → E sayfalarına gitti, sonra E'den D'ye, D'den B'ye döndü ve B'den C'ye geçti. Vekil önbellek kullanıldığına göre sunucu kaydında ne görünür ve eksik istekler nasıl bulunur?

> **Çözüm (derste verildi):** D ve B'ye dönüşler önbellekten karşılandığı için kayıtta görünmez: kayıt $A \to B \to D \to E \to C$ olur. Sitenin bağlantı yapısına bakılarak eksik istekler eklenir ve yol $A \to B \to D \to E \to D \to B \to C$ olarak tamamlanır.

Dersin Toparlaması

Bu derste bağlantı analizi bölümü HITS algoritmasıyla tamamlandı. PageRank sorgudan bağımsız, önceden hesaplanan bir popülerlik skoruydu. HITS ise sorguyla eşleşen kök kümeyi bir adım ileri ve geri genişleterek taban kümeyi kurar, bu küçük graf üzerinde her sayfa için birbirini besleyen otorite ve merkez skorlarını hesaplar. Skorlar $a = L^T L\, a$ ve $h = L\, L^T h$ denklemlerinin kuvvet yinelemesiyle bulunur. Buradaki $L^T L$ ortak atıf, $L\, L^T$ bibliyografik eşleşme matrisidir. HITS sorguya bağlı sıralama yapar, ama manipülasyona açıktır, konu kayması yaşayabilir ve hesap sorgu anında yapıldığı için PageRank'ten daha az verimlidir.

Dersin ikinci yarısında web madenciliğinin üçüncü kolu olan web kullanım madenciliğine geçildi. Amaç, sunucu ve uygulama kayıt dosyalarındaki tıklama akışından kullanıcıların davranış örüntülerini çıkarmaktır. Bunun için veri birleştirilir ve temizlenir, kullanıcılar (çerez, oturum açma, yazılım ajanı, gömülü oturum kimliği ya da IP + ajan ile) tanımlanır, oturumlar (zamana ya da gezinmeye dayalı sezgilerle) bulunur, sayfa görüntülemeleri belirlenir, önbellek nedeniyle eksik kalan yollar tamamlanır ve bölümler tanımlanır. Sonuçta oluşan kullanıcı işlem veritabanı üzerinde dönem boyunca görülen birliktelik kuralı, sıralı örüntü, kümeleme ve sınıflandırma teknikleri uygulanır. Bir sonraki hafta dönemin son haftasıdır ve proje sunumlarına ayrılmıştır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.